Assignment 3: Technical Implementation Report

AgroLedger: Technical Implementation Report

Project Overview:

AgroLedger is a blockchain-based solution developed for the Canadian Sheep Federation (CSF) to provide end-to-end traceability of sheep within the supply chain. The platform enhances transparency and accountability by tracking key lifecycle events of sheep, including registration, movement, and slaughter. The system is built on **Hyperledger Fabric**, with IoT simulation via API calls to demonstrate real-time data collection and integration, along with **QR code generation** and **IPFS** for storing records.

---

1. System Architecture:

AgroLedger utilizes a modular system design that integrates blockchain technology, a backend API layer, and a web-based user interface. The architecture is composed of the following components:

1.1. Blockchain:

- **Platform:** Hyperledger Fabric
- **Role:** The blockchain stores immutable records of sheep lifecycle events, ensuring data integrity across the supply chain.
- **Smart Contracts (Chaincode):** Developed in JavaScript, the smart contracts automate sheep registration, movement tracking, and other lifecycle events, ensuring transparency and security.

1.2. API Layer:

- **Backend:** Node.js
  The backend processes API requests and interfaces with the blockchain to retrieve and update records.
- **API Calls for IoT Simulation:**
  API calls simulate the transmission of data (e.g., temperature, location) that mimics IoT inputs, demonstrating how real-world data would interact with the blockchain.

1.3. Front-End:

- **Framework:** React.js
  Provides a user-friendly interface for stakeholders to interact with the blockchain, enter and view sheep data, and simulate IoT-driven events.
- **Role-Based Access Control (RBAC):**
  Ensures that users can only access the data and actions relevant to their roles (e.g., producer, transporter, abattoir).

1.4. Database:

- **Technology:** MySQL
  Stores supplementary data that does not require blockchain immutability, such as user management and event logs.
- **ORM:** Prisma is used to manage interactions with the database, simplifying backend operations.

1.5. QR Code Implementation:

- **QR Code Generation:**
  Each sheep registered in the system is assigned a unique QR code, which can be scanned by stakeholders (e.g., producers, transporters, consumers) to view relevant lifecycle data.
- **Data Display via QR Code:**
  Scanning a QR code allows users to access the blockchain record for that sheep, including registration details, movement history, and

current status.

1.6. IPFS (InterPlanetary File System):

- **Decentralized Storage:**
  Critical documents and records (e.g., sheep health certificates, movement reports) are stored on IPFS to ensure decentralized, secure, and easily retrievable records.
- **IPFS Integration with Blockchain:**
  A link to the IPFS hash is stored on the blockchain, ensuring that files can be accessed securely and immutably.

---

2. Smart Contracts and Permissions:

2.1. Smart Contracts:

The AgroLedger system is underpinned by smart contracts that automate critical sheep lifecycle events. The key smart contracts include:

- **Sheep Registration:** Enables producers to register sheep by assigning unique identifiers.
- **Movement Tracking:** Logs the movement of sheep between locations, ensuring traceability throughout the supply chain.
- **Slaughter Event Logging:** Captures final lifecycle events when sheep are processed at the abattoir.

2.2. Role-Based Access Control (RBAC):

Different user roles are defined, with each role granted specific permissions:

- **Producers:** Can register sheep and view movement data.
- **Transporters:** Update sheep movement data as they move between farms and slaughterhouses.
- **Abattoirs:** Log the final slaughter events.
- **Program Administrators:** Oversee the entire system, managing user roles and permissions.

Each role interacts with the smart contracts in ways specific to their responsibilities, ensuring access control and data security.

---

3. IoT Simulation:

In AgroLedger, IoT functionality is demonstrated through simulated API calls rather than real IoT devices. This allows stakeholders to visualize how data collected by IoT devices would integrate with the blockchain in real-time:

3.1. API-Driven IoT Simulation:

- **Temperature Simulation:** API endpoints simulate the transmission of temperature data as if collected by sensors attached to sheep. This data is submitted to the backend, which updates the blockchain.
- **Location Simulation:** GPS data is also simulated through API calls, demonstrating how the movement of sheep would be recorded in real time, ensuring visibility for stakeholders.

3.2. Front-End Integration:

- The front-end interface displays the simulated IoT data, allowing users to see how real-time monitoring would appear in production. This includes charts and logs of temperature and location data for individual sheep.

---

4. User Interface (UI) Development:

4.1. Front-End Implementation:

- **Framework:** React.js
- **Key Features:**

- **Forms for Data Entry:** Enables users to input sheep registration, movement, and slaughter data.
- **Simulated IoT Data:** Displays IoT-like data, including temperature and location, updated through API calls.
- **QR Code Scanning:** Users can scan QR codes to view lifecycle data of specific sheep.

4.2. User Experience (UX):

- **Responsive Design:** The UI is designed to work seamlessly across devices, including desktops, tablets, and mobile phones.
- **Interactive Forms:** Forms are powered by Formik, allowing for smooth data entry and validation.

---

5. Security and Data Integrity:

5.1. Authentication and Authorization:

- **Certificate Authority (CA) and Role-Based Access Control (RBAC):**
  Hyperledger Fabric's CA is used to issue digital certificates to authenticate users and assign them roles. This ensures only authorized users can access the blockchain and perform actions based on their role, enforcing RBAC.
  - **RBAC Implementation:** Users are issued digital certificates that grant specific permissions based on their role. This ensures users can only interact with the blockchain in ways that align with their responsibilities.
- **JWT (JSON Web Tokens):**
  For API-based interactions, AgroLedger utilizes JWTs to authenticate users. After successful login, a JWT is issued and included in the header of each API request for secure communication.
  - **Token-Based Authentication:** The JWT encodes user-specific information, such as their role and permissions, ensuring that only authorized API calls are allowed.
  - **Secure API Calls:** JWTs are validated with every API request to ensure that the user has appropriate permissions for the requested action.

5.2. Blockchain Security:

- **Immutability:** Once a transaction is recorded on the blockchain, it cannot be altered, ensuring data integrity.
- **Privacy:** Hyperledger Fabric's permissioned network ensures that only authorized users can view or interact with specific data.

---

6. Technical Challenges and Solutions:

6.1. Technical Challenges:

- **Smart Contract Development:**
  Defining complex interactions, such as multiple stakeholders updating the same sheep lifecycle data, while maintaining data consistency.
- **Simulating IoT Data:**
  Demonstrating the functionality of IoT without access to physical devices, which required simulating real-time data inputs.
- **User Management and Certificate Assignment:**
  Managing the addition of users to the blockchain network from the backend, including the secure assignment of certificates, authentication, and role assignment.
- **QR Code and IPFS Integration:**
  Implementing a system to generate unique QR codes for each sheep and integrating decentralized storage (IPFS) to store critical records.

6.2. Solutions:

- **Smart Contracts:**
  Developed smart contracts that allow easy updates and additions without disrupting ongoing operations.
- **API-Based IoT Simulation:**
  API endpoints were used to simulate IoT data, allowing us to demonstrate real-time data interaction without hardware dependencies.

- **Automated User Management:**
  Integrated Hyperledger Fabric's CA to automate user management. New users are securely added to the network, and digital certificates are issued and assigned based on their roles.
- **QR Code Generation and IPFS:**
  QR codes were generated for each sheep, enabling stakeholders to scan and view lifecycle data. Critical records are stored on IPFS, with the hash linked to the blockchain for secure access and retrieval.

---

7. Conclusion and Future Work:

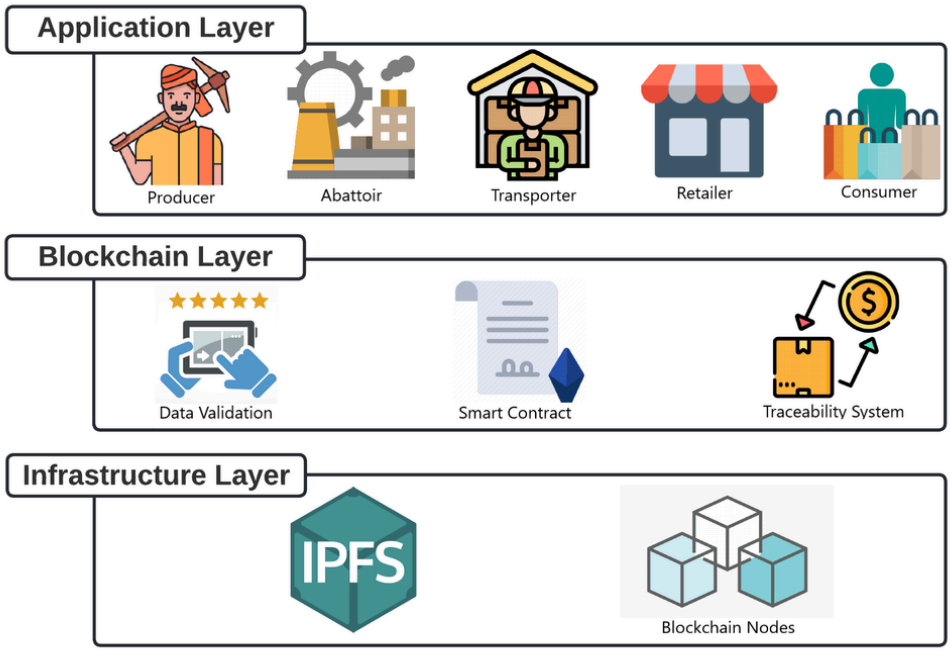
The AgroLedger system has successfully met the primary goals of the project, providing traceability and transparency in the sheep supply chain using blockchain technology, with simulated IoT integration and decentralized storage through IPFS.

Next Steps:

- **Scaling Up:** Expand the system to support additional livestock types.
- **Data Analytics:** Implement predictive analytics for livestock health and performance.
- **Physical IoT Integration:** Eventually replace simulated API data with real IoT devices for live data collection.

Appendices:

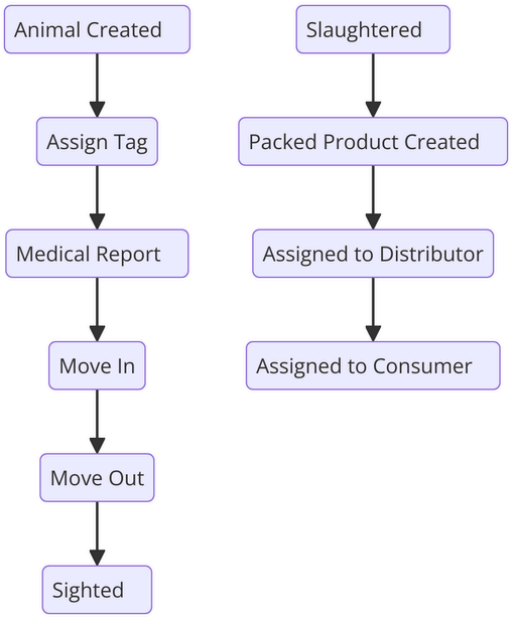
- **Appendix A:** System Architecture Diagram



- **Appendix B:** API Documentation

Swagger- API D…ed).pdf
19 Sep 2024, 03:12 PM

- **Appendix C:** Smart Contract Flow



This report provides a comprehensive view of the technical implementation of the AgroLedger system, including blockchain integration, API-driven IoT simulation, QR code generation, and IPFS storage.